INSTITUTO TECNOLÓGICO SUPERIOR DE XALAPA
Ingeniería en Sistemas Computacionales | SCC-1017 Métodos Numéricos

REPORTE TÉCNICO DE PRÁCTICA DE INTERPOLACIÓN

Práctica: Interpolación de Lagrange vs. Diferencias Divididas de Newton (Unidad 5)

Docente: Mtro. Víctor Hugo Vásquez Herrera

Estudiante: Francisco Javier Hernandez Suarez

1. ENTREGA DE CÓDIGO Y REPOSITORIO GITHUB

El código fuente correspondiente a la implementación completa en Java de ambos métodos de interpolación polinomial se encuentra alojado en el repositorio público personal:

- **Enlace al Repositorio GitHub:** https://github.com/javier24viper/-Interpolaci-ndeLagrangevs.DiferenciasDivididasdeNewton.git
- **Estructura del Proyecto:** El repositorio cuenta con el archivo obligatorio `README.md` como guía rápida de requerimientos de compilación (JDK 11+), descripción general y pasos de ejecución en consola.

2. INTRODUCCIÓN Y MARCO TEÓRICO

La **interpolación polinomial** es una técnica matemática indispensable en el análisis numérico que consiste en construir una función polinomial continua de grado a lo más $n - 1$ que pasa exactamente por un conjunto de n puntos o nodos dados (x_0, y_0), (x_1, y_1), ..., (x_{n-1}, y_{n-1}). Existen diversos enfoques para la obtención de dicho polinomio de interpolación unico.

Polinomios de Interpolación de Lagrange

El método de Lagrange formula el polinomio interpolante $P(x)$ directamente como una combinación lineal de los valores conocidos y_i ponderados por los llamados **polinomios base de Lagrange** $L_i(x)$:

$$P(x) = \sum_{i=0}^{n-1} y_i \cdot L_i(x)$$

Donde el polinomio base $L_i(x)$ se define mediante la productoria:

$$L_i(x) = \prod_{j=0,\, j \neq i}^{n-1} (x - x_j) / (x_i - x_j)$$

Este método destaca por su elegancia y concisión matemática, pues no requiere la solución de un sistema de ecuaciones lineales complejas ni almacenamiento intermedio en matrices.

Diferencias Divididas de Newton

El método de Newton expresa el polinomio interpolante en forma Newtoniana mediante diferencias divididas finitas de orden creciente:

$$P(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots$$

Las diferencias divididas se calculan de manera recursiva a partir de la tabla o matriz triangular de diferencias divididas, dada por:

$$f[x_i, \dots, x_{i+k}] = (f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]) / (x_{i+k} - x_i)$$

3. DESARROLLO DEL CÓDIGO Y CORRECCIÓN DE LAGRANGE

Para la clase `InterpolacionLagrange.java`, se identificó la rutina incompleta en la acumulación de la productoria del polinomio base $L_i(x)$ dentro de los ciclos `for` anidados. La línea errónea original `termino *= 0.0;` impedía la multiplicación acumulativa de las fracciones.

```
Línea corregida en InterpolacionLagrange.java (Línea 31):
termino *= (xEval - x[j]) / (x[i] - x[j]);
```

Justificación del código: En cada iteración del bucle secundario `j` (con la restricción `i != j`), se evalúa el factor individual `(xEval - x[j]) / (x[i] - x[j])` y se multiplica de forma acumulativa con la variable `termino` (la cual inicia con el valor de `y[i]`). Al finalizar el bucle interno, `termino` almacena el valor final de $y_i \cdot L_i(x_{eval})$, el cual se suma directamente al acumulador global `resultado`.

4. PRUEBAS NUMÉRICAS Y RESULTADOS EN CONSOLA

Para la verificación experimental de los programas, se definió el siguiente conjunto de datos y punto de evaluación a libre elección:

- **Nodos de entrada:** X = [1.0, 2.0, 4.0, 5.0], Y = [2.0, 1.0, 5.0, 10.0]
- **Punto de evaluación (xEval):** 3.0

Salida de Ejecución del Programa en Consola

```
=== PRUEBA DE INTERPOLACIÓN ===
Nodos X: [1.0, 2.0, 4.0, 5.0]
Nodos Y: [2.0, 1.0, 5.0, 10.0]
Punto a evaluar (xEval): 3.0

--- MÉTODO DE LAGRANGE ---
Resultado interpolado P(3.0) = 2.000000

--- MÉTODO DE DIFERENCIAS DIVIDIDAS DE NEWTON ---
Tabla / Matriz de Diferencias Divididas:
i=0 | x=1.0 | y= 2.0000 | -1.0000 |  1.6667 | -0.4167 |
i=1 | x=2.0 | y= 1.0000 |  2.0000 |  1.0000 |
i=2 | x=4.0 | y= 5.0000 |  5.0000 |
i=3 | x=5.0 | y=10.0000 |

Coeficientes del Polinomio (Diagonal Superior):
a0 = 2.0000, a1 = -1.0000, a2 = 1.6667, a3 = -0.4167

Resultado interpolado P(3.0) = 2.000000
Diferencia entre métodos: 0.000000e+00 (Coincidencia exacta)
```

5. ANÁLISIS COMPARATIVO Y PREGUNTAS DE DISCUSIÓN

1. **¿Ambos métodos entregan exactamente el mismo resultado para el punto evaluado? Justifique numéricamente.**
   Respuesta: Sí, ambos métodos entregan exactamente el mismo valor (`2.000000`). Esto se justifica analíticamente mediante el *Teorema de Unicidad del Polinomio Interpolante* **Burden & Faires (2011)**, el cual establece que dado un conjunto de *n* puntos distintos, existe únicamente un solo polinomio de grado a lo más *n − 1* que pasa por todos ellos. Lagrange y Newton son simplemente dos formulaciones algebraicas distintas del mismo objeto matemático básico.
2. **Si se añade un nuevo punto al conjunto de datos iniciales, ¿cuál de los dos métodos exige un menor esfuerzo computacional para recalcular la interpolación y por qué?**
   Respuesta: El método de **Diferencias Divididas de Newton** exige un esfuerzo computacional significativamente menor. Debido a su estructura incremental, al incorporar un nuevo nodo (x_n, y_n), los coeficientes calculados previamente permanecen inalterados y únicamente se requiere agregar una nueva fila en la tabla de diferencias divididas. En contraste, en el método de Lagrange se alteran todos los denominadores ($x_i - x_j$) de los polinomios base $L_i(x)$, lo que obliga a recalcular la productoria completa desde cero **Chapra & Canale (2015)**.
3. **¿Qué complejidad algorítmica (O(n)) presenta la evaluación de cada uno de los dos algoritmos?**
   Respuesta:

4.
    - **Lagrange:** Complejidad **$O(n^2)$** operaciones para la evaluación directa, ya que requiere calcular *n* polinomios base, cada uno con *n − 1* multiplicaciones.
    - **Diferencias Divididas de Newton:** La construcción de la tabla de diferencias divididas requiere **$O(n^2)$** operaciones. Sin embargo, una vez calculada la tabla, la evaluación del polinomio utilizando la regla de Horner requiere únicamente **O(n)** multiplicaciones y adiciones.

6. CONCLUSIONES

En el desarrollo de esta práctica se comprobó de manera práctica la equivalencia matemática entre los métodos de interpolación de Lagrange y Newton. La programación en Java permitió comprender que, aunque Lagrange posee una estructuración de código sumamente intuitiva mediante loops de productoria, Newton ofrece ventajas algorítmicas contundentes en escenarios reales de computación donde el número de puntos no es fijo o se requiere evaluar iterativamente una tabla dinámica. La correcta implementación de los controles de validación en Java asegura la robustez numérica evitando divisiones entre cero por nodos duplicados.

7. REFERENCIAS BIBLIOGRÁFICAS

- Burden, R. L., & Faires, J. D. (2011). *Análisis Numérico* (9a. ed.). Cengage Learning.
- Chapra, S. C., & Canale, R. P. (2015). *Métodos Numéricos para Ingenieros* (7a. ed.). McGraw-Hill Education.
- Mathews, J. H., & Fink, K. D. (2004). *Métodos Numéricos con MATLAB* (4a. ed.). Pearson Educación.